
FULL-STACK ENGINEERING PROJECT REPORT • SPRING BOOT & REACT
MODULE: J2EE & SPRING FRAMEWORKS

LibraCore

Smart Library Management Platform

Full-Stack Web Application
React • Spring Boot • MySQL

Prepared by

Maha El Allam

Fatima-Ezzahrae Ziyter

Supervised by

Professor Amri

Academic Year 2025 – 2026

Contents

1	Introduction	3
1.1	Project Rationale	3
1.2	Core Capabilities	3
1.3	System Architecture Overview	4
2	Project Objectives	5
2.1	Core Functional Objectives	5
2.2	Technical Quality Objectives	5
2.3	Expected Outcomes	6
3	Technologies Used	7
3.1	Technical Stack Overview	7
3.2	Architectural Highlights	7
4	System Architecture	9
4.1	Architectural Layers	9
4.2	Request Processing Flow	9
4.3	Data Transformation Pattern	10
4.4	Communication Protocol	10
5	Backend Design	11
5.1	Domain Model and Entities	11
5.1.1	User	11
5.1.2	Role	11
5.1.3	Book (Livre)	11
5.1.4	Category (Categorie)	12
5.1.5	Book Details (DetailsLivre)	12
5.2	Repository Layer	12
5.3	Service Layer	12
5.4	Controller Layer	13
5.5	Data Transfer Objects (DTOs)	13
5.6	Mapping Layer	14
5.7	Global Exception Handling	14
6	Security Implementation	15
6.1	JWT Authentication Workflow	15
6.2	Role-Based Access Control (RBAC)	15
6.3	Security Infrastructure Configuration	16

7	Frontend Implementation	17
7.1	Core Modules	17
7.2	User Interface Showcase	17
8	REST API Endpoints	19
8.1	Authentication	19
8.2	Book Management	19
8.3	Category Management	19
8.4	User Administration	20
8.5	Export and Reporting	20
9	Export and Reporting Features	21
9.1	Overview	21
9.2	Implementation Details	21
9.3	Export Workflow	22
10	Testing Strategy	23
10.1	Testing Frameworks and Environment	23
10.2	Unit Tests — Service Layer Isolation	23
10.3	Integration Tests — Controller Layer Validation	24
10.4	Test Coverage Summary	24
11	Results and Implementation	25
11.1	Functional Validation	25
11.2	Interface and User Experience	25
11.3	Technical Validation	26
12	Conclusion and Perspectives	27
12.1	Key Achievements	27
12.2	Future Roadmap	27

Chapter 1

Introduction

In the contemporary digital era, the efficient management of information resources is a critical operational requirement for academic and public institutions. Manual library administration processes are increasingly inadequate in the face of growing collections, complex user hierarchies, and the demand for real-time reporting. The **LibraCore Library Management System** (LMS) presented in this report addresses these challenges through a modern, full-stack web application engineered to centralize, automate, and streamline the administration of library resources.

1.1 Project Rationale

Traditional library operations suffer from fragmented data management, limited access control, and a lack of analytical visibility. This system resolves these issues by providing a unified digital environment where librarians and administrators can manage books, categories, and users with precision and efficiency. The platform bridges the gap between raw data storage and actionable operational insights, transforming routine administrative workflows into streamlined, automated processes.

1.2 Core Capabilities

The platform is designed around three foundational pillars, each addressing a distinct dimension of library administration:

- **Identity and Access Management:** A stateless, token-based authentication mechanism enforces granular Role-Based Access Control (RBAC), ensuring that each user class interacts exclusively with the resources appropriate to their clearance level.
- **Resource Administration:** Comprehensive CRUD operations enable the precise cataloging of books and their organization into distinct, manageable categories, supported by relational integrity constraints.
- **Data Analytics and Portability:** Real-time statistical dashboards provide administrative visibility into library usage, while automated export features generate professional PDF and Excel reports for institutional reporting.

1.3 System Architecture Overview

The application adopts a robust, decoupled client-server architecture. The backend is implemented as a RESTful API, ensuring high availability and standardized data exchange, while the frontend delivers a responsive and user-centric interface. This separation of concerns guarantees independent evolution of the client and server components, facilitating maintainability and long-term scalability. The subsequent chapters detail the technical decisions, design patterns, and implementation strategies that underpin this architecture.

Chapter 2

Project Objectives

Building upon the operational context established in the introduction, this chapter formalizes the system's objectives across both functional and technical dimensions. The primary goal of this project is to develop a secure, high-performance, and scalable Library Management System that centralizes data management while delivering advanced reporting and administrative capabilities.

2.1 Core Functional Objectives

The system is organized around three functional pillars that collectively define its operational scope:

- **Resource Management:** Implementation of robust CRUD operations for books, ensuring data integrity through relational mapping between entities and category-based organizational structures.
- **Identity and Access Management (IAM):** Deployment of a secure authentication module using JSON Web Tokens (JWT), coupled with granular Role-Based Access Control (RBAC) to enforce distinct administrative and user-specific operational boundaries.
- **Business Intelligence and Data Portability:** Provision of analytical dashboards and automated document generation in PDF and Excel formats, supporting data-driven administrative decision-making.

[Image of library management system architecture diagram]

2.2 Technical Quality Objectives

Beyond core functionality, the project targets specific non-functional quality attributes that govern the system's long-term viability:

Attribute	Description
Reliability	Consistent data integrity and comprehensive error handling across all API endpoints.
Security	Resource protection through stateless JWT authentication and structured filter chains.
Usability	An intuitive, responsive, and cross-platform frontend interface.
Maintainability	Adherence to professional standards through DTO patterns, mapper abstractions, and layered service architecture.

Table 2.1: Non-Functional Quality Objectives

2.3 Expected Outcomes

The final system aims to replace manual library processes with a fully automated solution, significantly reducing administrative overhead and improving the operational experience for both staff and patrons. These objectives provide the criteria against which the implementation is evaluated throughout this report.

Chapter 3

Technologies Used

The application is built on a modern, industry-standard technology stack selected for its robustness, community support, and suitability for enterprise-grade web development. Each technology was chosen to address a specific architectural concern, from data persistence to user interface rendering.

3.1 Technical Stack Overview

Category	Technologies
Frontend	React.js (Vite build tool), Axios for API integration, React Router for SPA navigation, Recharts for data visualization, CSS3 .
Backend	Spring Boot 3 (core framework), Spring Security and JWT (authentication layer), MapStruct (DTO mapping), Lombok (boilerplate reduction).
Data Access	Spring Data JPA / Hibernate (ORM), MySQL (relational database).
Utilities	Swagger/OpenAPI (API documentation), Apache POI (Excel export), iText 7 (PDF generation).

Table 3.1: Full-Stack Technology Overview

3.2 Architectural Highlights

[Image of full-stack web application development stack]

The technology selections reflect several deliberate architectural decisions:

- **Decoupled Architecture:** The frontend and backend communicate exclusively through a RESTful API, enabling independent development, deployment, and scaling of each tier.
- **Security-First Design:** Stateless JWT authentication eliminates server-side session management, while RBAC ensures that resource access is strictly governed by user

role.

- **Data Mapping Efficiency:** MapStruct generates type-safe mapping code between JPA entities and DTOs at compile time, preventing data leakage while eliminating repetitive boilerplate.
- **Dynamic Reporting:** The integration of iText 7 and Apache POI enables on-the-fly generation of professional reports directly from live system data, supporting the platform's business intelligence capabilities.

With the technology stack established, the following chapter describes how these components are organized into a coherent system architecture.

Chapter 4

System Architecture

The application follows a **Three-Tier Architecture** that enforces a clear separation between the presentation layer, the business logic layer, and the data access layer. This architectural pattern improves maintainability, scalability, security, and code reusability across the entire system.

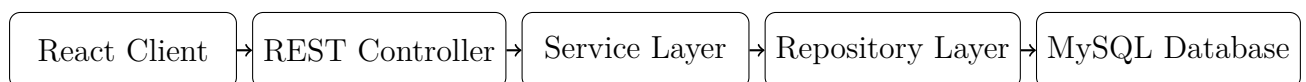
4.1 Architectural Layers

The system is composed of three primary tiers, each with a distinct responsibility:

- **Presentation Layer (React.js)**: Responsible for the user interface, client-side navigation, state management, and communication with the backend via REST APIs.
- **Business Logic Layer (Spring Boot)**: Handles business rules, authentication, authorization, input validation, and the orchestration of client requests through the service and controller components.
- **Data Access Layer (MySQL)**: Stores all persistent application data, accessed exclusively through Spring Data JPA repositories, ensuring a clean abstraction over the relational database.

4.2 Request Processing Flow

Every client interaction follows a structured processing pipeline. The diagram below illustrates the path of a typical HTTP request through the system:



The client dispatches HTTP requests to the REST controllers. Controllers delegate processing to the service layer, which applies business rules and coordinates with the repositories. Repositories interact with the MySQL database to retrieve or persist data, before results propagate back through the chain to the client as a structured JSON response.

4.3 Data Transformation Pattern

To ensure a rigorous separation between internal domain entities and the external API surface, the application employs **Data Transfer Objects (DTOs)** in conjunction with **MapStruct** for automated conversion.

Layer	Component	Data Type
Controller	REST API Request / Response	DTO
Mapper	Bidirectional Conversion	DTO $\leftrightarrow$ Entity
Service	Business Logic Processing	Entity
Repository	Persistence Operations	Entity

DTOs prevent direct exposure of the JPA entity structure through the API, resolve circular reference issues inherent in bidirectional relationships, and minimize the volume of transmitted data. MapStruct generates the mapping implementation at compile time, yielding both type safety and performance efficiency.

4.4 Communication Protocol

Frontend and backend components communicate through a RESTful API using **JSON over HTTPS**. This approach provides platform independence, secure communication, and straightforward integration with third-party systems. All responses adhere to standard HTTP status conventions, with structured JSON payloads ensuring consistency throughout the system. The following chapters examine the backend design in detail, beginning with the domain model and progressing through each architectural layer.

Chapter 5

Backend Design

The backend is organized into distinct horizontal layers—entities, repositories, services, controllers, DTOs, and mappers—each with a well-defined responsibility. This chapter examines each layer in sequence, illustrating how they collaborate to deliver the system’s core functionality.

5.1 Domain Model and Entities

The application’s domain model is implemented using Jakarta Persistence (JPA), mapping object-oriented class hierarchies to relational database tables through an ORM approach.

5.1.1 User

The **User** entity represents authenticated system principals and manages their credentials. It carries a unique **username** and an encoded **password**, and maintains a **ManyToMany** relationship with the **Role** entity using an **EAGER** fetch strategy, ensuring that role assignments are always available during the authentication and authorization workflow.

5.1.2 Role

The **Role** entity defines security levels within the system through a mapping to the **ERole** enumeration (**ADMIN**, **MANAGER**, **USER**). It maps directly to Spring Security authorities, enabling granular, type-safe permission management across the application.

5.1.3 Book (Livre)

As the central entity of the system, **Livre** captures the core bibliographic attributes: **id**, **titre**, and **isbn**. It maintains a **ManyToOne** relationship with **Categorie** and a **OneToOne** relationship with **DetailsLivre**, configured with **CascadeType.ALL** to ensure that supplementary technical details remain synchronized with the primary record throughout the entity lifecycle.

5.1.4 Category (Categorie)

The `Categorie` entity organizes books into thematic groups through a `OneToMany` relationship with `Livre`. The `@JsonIgnore` annotation is applied to this collection to prevent circular reference issues during API serialization, a critical consideration given the bidirectional nature of the relationship.

5.1.5 Book Details (DetailsLivre)

`DetailsLivre` stores granular metadata that extends the primary `Livre` record, including `auteur`, `nombrePages`, and `emplacementRayon`. Linked via a `OneToOne` join column, this separation maintains a clear distinction between core bibliographic data and physical book specifications, supporting a cleaner domain model.

5.2 Repository Layer

The Repository layer implements the Data Access Object (DAO) pattern, abstracting all database interactions through Spring Data JPA. By extending `JpaRepository`, each repository inherits standard CRUD operations while providing custom queries for domain-specific requirements.

- **LivreRepository:** Implements a JOIN FETCH query (`findAllWithDetails`) to resolve the N+1 problem during bulk book retrieval, and provides an aggregation query (`countLivresParCategorie`) to support the statistical reporting dashboard.
- **UserRepository:** Manages user persistence and exposes a `findByUsername` method essential to the authentication workflow.
- **RoleRepository:** Provides an `Optional<Role>` lookup based on the `ERole` enumeration, ensuring type-safe role resolution during user registration and profile updates.
- **CategorieRepository:** Extends standard JPA operations with a custom count query supporting application-wide statistics.
- **DetailsLivreRepository:** Manages the persistence of book metadata while maintaining relational integrity between `Livre` and its associated technical details.

The consistent use of Spring Data's `Optional` wrapper for all single-entity lookups significantly reduces null-pointer risks and reinforces the robustness of the data access logic.

5.3 Service Layer

The Service layer constitutes the core of the application's business logic, mediating between the Controller and Repository layers while enforcing domain rules and data consistency constraints. Each service is annotated with `@Transactional` where required to guarantee atomic execution.

- **LivreService:** Manages the full lifecycle of book records, including entity mapping, validation of associated `Categorie` entities prior to persistence, and the maintenance of bidirectional consistency between `Livre` and `DetailsLivre`.

- **UserService**: Orchestrates user management operations, including secure password encoding via `PasswordEncoder`, role-to-enumeration conversion, and existence validation, ensuring robust profile creation and update processes.
- **CategorieService**: Provides full CRUD management for categories, with explicit protection against overwriting existing book-to-category associations during update operations, preserving referential integrity.
- **Security Services (`UserDetailsServiceImpl`, `UserDetailsImpl`)**: `UserDetailsServiceImpl` retrieves user credentials from the database to support Spring Security’s authentication workflow, while `UserDetailsImpl` structures user data—including ID, username, and granted authorities—for JWT generation and session context population.

All services employ the custom `ResourceNotFoundException` to deliver precise, actionable error feedback to calling controllers when requested resources cannot be located.

5.4 Controller Layer

The Controller layer serves as the RESTful entry point of the API, coordinating the translation of HTTP requests into service invocations and returning structured responses to the client. All controllers are annotated with `@CrossOrigin("*")` to support seamless frontend integration.

- **AuthController**: Manages the `/signin` and `/signup` endpoints, integrating with Spring Security’s `AuthenticationManager` and `JwtUtils` to produce and return valid JWT tokens upon successful authentication.
- **LivreController**: Exposes CRUD endpoints for the book catalog, employing `LivreMapper` to translate between `Livre` entities and `LivreDTO` instances, ensuring that internal model details are never leaked through the API surface.
- **CategorieController**: Manages category endpoints with Jakarta Validation applied at the request level, ensuring data integrity before processing reaches the service layer.
- **AdminController**: Provides privileged endpoints for user management—listing, creating, updating, and deleting users—restricted to roles with the appropriate authority level.
- **ExportController**: Delivers data portability through endpoints for PDF report generation (iText 7), Excel spreadsheet export (Apache POI), and system statistics retrieval for dashboard consumption.

5.5 Data Transfer Objects (DTOs)

DTOs are implemented throughout the application to decouple the internal domain model from the external API representation, providing several structural benefits. They prevent exposure of the complete JPA entity graph, resolve circular reference issues inherent in bidirectional `Livre`–`Categorie` relationships, and restrict transmitted data to only what is required by each endpoint.

- **Authentication DTOs** (`LoginRequest`, `SignupRequest`, `JwtResponse`): Encapsulate authentication-related data, ensuring that credentials and tokens are never commingled with domain entities.
- **LivreDTO**: Incorporates Jakarta Validation constraints (`@NotBlank`, `@Size`, `@NotNull`) to enforce data integrity before requests reach the service layer.
- **UserDTO**: Facilitates secure user profile manipulation without exposing sensitive fields such as hashed passwords to API consumers.

5.6 Mapping Layer

The mapping layer is managed by **MapStruct**, configured with `componentModel = "spring"` to integrate seamlessly with Spring's dependency injection framework. The **LivreMapper** automates bidirectional conversion between **Livre** entities and **LivreDTO** instances, centralizing transformation logic and eliminating manual mapping errors during complex object transitions.

5.7 Global Exception Handling

The application employs a centralized error handling strategy through a `@RestControllerAdvice`-annotated **GlobalExceptionHandler**, replacing generic server error responses with standardized, human-readable HTTP responses.

- **Resource Not Found**: The `ResourceNotFoundException` triggers a 404 Not Found response whenever a requested book, user, or category cannot be located.
- **Constraint Violations**: Database constraint violations—including foreign key conflicts and unique constraint breaches identified by `UK_` prefixes—are intercepted and returned as 409 Conflict responses.
- **Standardized Error Structure**: Every error response adheres to a consistent schema comprising a timestamp, HTTP status code, and descriptive message, simplifying both frontend integration and system debugging.

Chapter 6

Security Implementation

Security is a first-class architectural concern in this system. The entire security infrastructure is built upon Spring Security, leveraging a stateless JWT-based authentication mechanism to ensure secure, scalable API communication between the React frontend and the Spring Boot backend.

6.1 JWT Authentication Workflow

The authentication process operates statelessly, eliminating server-side session storage and enabling horizontal scalability. The workflow proceeds through the following stages:

1. **Credential Submission:** The user submits credentials via `AuthController`, which delegates validation to the `AuthenticationManager`.
2. **User Validation:** `UserDetailsService` retrieves the user record from the database, and `UserDetails` structures it as a Spring Security principal.
3. **Token Generation:** Upon successful authentication, `JwtUtils` generates a signed JWT containing the subject (username) and an expiration claim.
4. **Token Delivery:** A `JwtResponse` object carrying the token is returned to the client for subsequent use in request headers.
5. **Request Interception:** On each subsequent request, the `AuthTokenFilter` extracts and validates the `Authorization: Bearer <token>` header via `JwtUtils`.
6. **Security Context Population:** If the token is valid, the user's authentication object is injected into the Spring Security context, granting access to protected endpoints for the duration of the request.

6.2 Role-Based Access Control (RBAC)

Access control is enforced at both the request and method levels using `@EnableMethodSecurity` and route-level configuration within `WebSecurityConfig`. The system defines three hierarchical roles:

- **ADMIN:** Full system privileges, encompassing user management via `AdminController` and all category and book operations.
- **MANAGER:** Elevated access to book and category management operations, without user administration capabilities.
- **USER:** Read-only access to library resources and export capabilities.

6.3 Security Infrastructure Configuration

The backend security infrastructure is configured through `WebSecurityConfig` and several supporting components:

- **Stateless Session Policy:** `SessionCreationPolicy.STATELESS` is enforced system-wide, ensuring that no server-side session state is maintained and that the system complies fully with RESTful architectural constraints.
- **Unauthorized Access Handling:** The `AuthEntryPointJwt` component intercepts unauthorized access attempts and returns a `401 Unauthorized` status, preventing unauthenticated users from receiving detailed system information.
- **CORS Configuration:** `CorsConfig` explicitly whitelists the frontend origins (`http://localhost:5173, 5174`) and permits the necessary headers—`Authorization` and `Content-Type`—to support JWT bearer token transmission in cross-origin requests.

Chapter 7

Frontend Implementation

The frontend is developed as a modular, responsive Single Page Application (SPA) using React 19, built with the Vite toolchain. This architecture ensures a fluid, app-like user experience across all devices, with client-side routing managed by React Router 7 and HTTP communication handled through Axios.

7.1 Core Modules

The frontend is organized into four principal functional modules, each corresponding to a key administrative domain:

- **Authentication:** Secure login and registration portals that handle JWT acquisition, storage, and attachment to outgoing API requests.
- **Analytical Dashboard:** A centralized administrative control center featuring real-time statistical charts rendered via Recharts and key navigation metrics derived from the `/api/export/stats` endpoint.
- **Resource Management:** Dedicated interfaces for full CRUD operations on books and categories, ensuring data consistency through form validation and real-time API synchronization.
- **User Administration:** A privileged management interface restricted to administrators, supporting account creation, role assignment, modification, and deletion.

7.2 User Interface Showcase

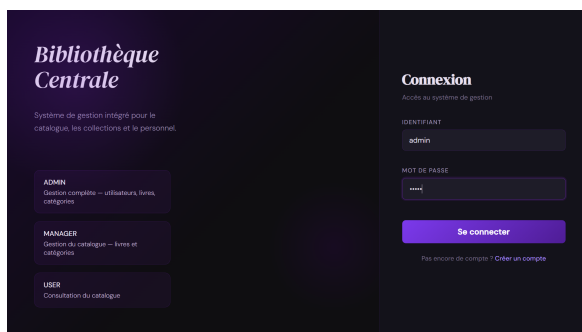


Figure 7.1: User Management Interface

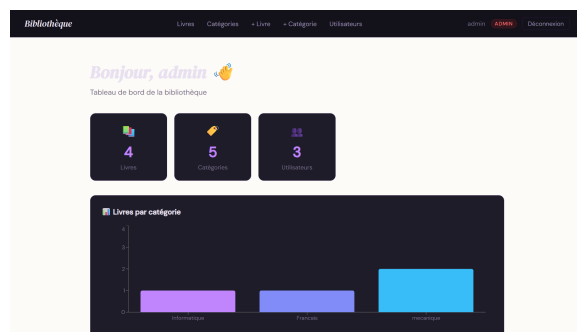


Figure 7.2: Analytical Dashboard

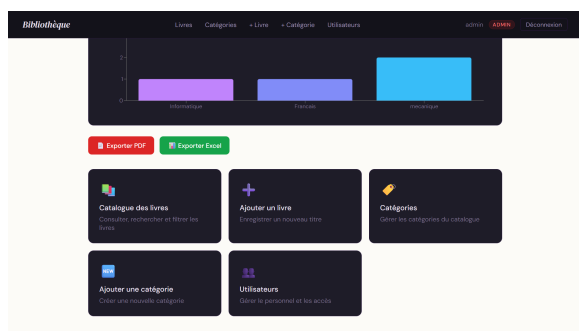


Figure 7.3: Book Management Interface

id	Identifiant	rang d'accès	actions administratives
#2	admin	ADMIN	Ajouter Modifier
#3	manager	MANAGER	Ajouter Modifier
#10	user	USER	Ajouter Modifier

Figure 7.4: Authentication Screen

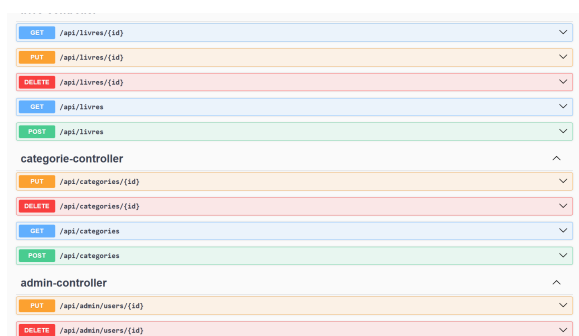


Figure 7.5: Swagger API Documentation

Chapter 8

REST API Endpoints

The API exposes a comprehensive set of endpoints organized by functional domain. All endpoints beyond the authentication routes are protected by JWT authentication and governed by role-based access control as defined in Chapter 6.

8.1 Authentication

Endpoint	Method	Description
/api/auth/signin	POST	User login; returns a signed JWT token upon successful authentication.
/api/auth/signup	POST	Public user registration endpoint.

8.2 Book Management

Endpoint	Method	Description
/api/livres	GET	Retrieve all books with details (USER+).
/api/livres	POST	Create a new book record (ADMIN / MANAGER).
/api/livres/{id}	GET	Retrieve a specific book by ID (USER+).
/api/livres/{id}	PUT	Update an existing book (ADMIN / MANAGER).
/api/livres/{id}	DELETE	Delete a book record (ADMIN / MANAGER).

8.3 Category Management

Endpoint	Method	Description
/api/categories	GET	List all categories (USER+).

/api/categories	POST	Create a new category (ADMIN / MANAGER).
/api/categories/{id}	PUT	Update category details (ADMIN / MANAGER).
/api/categories/{id}	DELETE	Remove a category (ADMIN / MANAGER).

8.4 User Administration

Endpoint	Method	Description
/api/admin/users	GET	Retrieve all registered system users (ADMIN only).
/api/admin/users?role={ROLE}	POST	Create a user with a specified role (ADMIN only).
/api/admin/users/{id}	PUT	Update user profile information (ADMIN only).
/api/admin/users/{id}	DELETE	Permanently remove a user from the system (ADMIN only).

8.5 Export and Reporting

Endpoint	Method	Description
/api/export/livres/pdf	GET	Stream all books as a formatted PDF report (USER+).
/api/export/livres/excel	GET	Stream all books as an Excel spreadsheet (USER+).
/api/export/stats	GET	Retrieve aggregated dashboard statistics (USER+).

Chapter 9

Export and Reporting Features

To facilitate data portability and administrative reporting, the system incorporates dedicated export capabilities. These features enable administrators and authorized users to generate professional-grade documents directly from the live system data, without any intermediate data extraction step.

9.1 Overview

The export layer leverages two well-established Java libraries, each selected for a specific document format and use case:

Format	Library	Primary Use Case
PDF	iText 7	High-fidelity, read-only institutional reports and official documentation.
Excel	Apache POI	Data analysis, pivot table preparation, and bulk data manipulation.

Table 9.1: Export Library Selection

9.2 Implementation Details

The export functionality is centralized within the **ExportController**, providing a secure and uniform mechanism for file generation. Several technical considerations informed the implementation:

- **Performance:** Efficient streaming techniques are employed to generate large files without excessive memory overhead, allowing the server to handle export requests without impacting other operations.
- **Dynamic Styling:**
 - **iText:** Custom styling via **DeviceRgb** color palettes and responsive table layouts ensures visual consistency across generated PDF documents.

- **Apache POI:** Advanced cell styling—including background fill patterns and automatic column sizing—is applied to Excel output for immediate readability.
- **Data Integrity:** All exports utilize the same service layer and DTOs as the REST API, guaranteeing that exported data is fully synchronized with the live system state.

9.3 Export Workflow

The generation process follows a clean three-phase cycle applicable to both formats:

1. **Fetch:** Data is retrieved from the repository using optimized queries—specifically, `JOIN FETCH` operations on `LivreRepository` to prevent N+1 performance degradation during bulk retrieval.
2. **Transform:** Retrieved entities are mapped to the target document structure, applying the appropriate formatting and layout rules.
3. **Stream:** The completed file is streamed directly to the HTTP response, minimizing temporary server-side storage and delivering the document to the client with minimal latency.

Chapter 10

Testing Strategy

A rigorous testing strategy was implemented to validate the reliability, security, and maintainability of the Library Management System. The strategy encompasses two complementary levels of testing—unit and integration—each targeting a distinct layer of the application and applying different isolation techniques.

10.1 Testing Frameworks and Environment

The testing infrastructure is built upon three core frameworks:

- **JUnit 5:** Provides the test execution engine and assertion library for both unit and integration tests.
- **Mockito:** Used to isolate service layers from their database dependencies in unit tests, enabling pure validation of business logic.
- **Spring Boot Test (@SpringBootTest):** Loads the full application context—including the security filter chain and MVC dispatcher—to simulate real HTTP interactions in integration tests.
- **H2 In-Memory Database:** Configured via `application-test.properties` under the `test` Spring profile, providing an isolated and reproducible data environment that does not affect production data.

10.2 Unit Tests — Service Layer Isolation

Unit tests are implemented for `LivreService` and `CategorieService` using the `@ExtendWith(MockitoEx` annotation. In this configuration, no Spring application context is loaded; instead, all repository dependencies are replaced with `@Mock` instances, and the class under test receives them via `@InjectMocks`. This approach tests business logic in complete isolation, with execution times measured in milliseconds.

The unit test suite covers 20 scenarios across the two service classes, including successful CRUD operations, exception propagation on missing resources, input validation edge cases, and bidirectional entity relationship management.

10.3 Integration Tests — Controller Layer Validation

Integration tests for `LivreController` and `CategorieController` employ `@SpringBootTest` with `@AutoConfigureMockMvc`, loading the complete Spring context including the JWT security filter chain. HTTP requests are simulated via `MockMvc`, with the service layer mocked using `@MockBean` to isolate the test from database state.

A key challenge resolved during this phase was the interaction between `@WithMockUser` and the application's custom authority model. Since `WebSecurityConfig` uses `.hasAnyAuthority("USER", "ADMIN", "MANAGER")` (without the `ROLE_` prefix), standard `@WithMockUser` annotations were replaced with `@WithMockUser(authorities = "USER")` and `@WithMockUser(authorities = "ADMIN")` to correctly simulate authenticated principals.

The integration test suite comprises 17 tests validating HTTP response codes (200, 400, 404), response body structure via JSON path assertions, and security boundary enforcement.

10.4 Test Coverage Summary

Class	Type	Tests	Coverage Focus
<code>LivreServiceTest</code>	Unit (Mockito)	12	Business logic, entity relationships, exception propagation
<code>CategorieServiceTest</code>	Unit (Mockito)	8	CRUD logic, association preservation, error handling
<code>LivreControllerTest</code>	Integration (<code>@SpringBootTest</code>)	9	HTTP routing, status codes, security enforcement
<code>CategorieControllerTest</code>	Integration (<code>@SpringBootTest</code>)	8	HTTP routing, validation, role-based access
Total		37	

Table 10.1: Test Coverage Summary

All 37 tests pass successfully, confirming that the service business logic and REST API layer behave correctly under both normal operation and error conditions.

Chapter 11

Results and Implementation

The development of this Library Management System has produced a functional, secure, and user-centric platform that meets the objectives defined at the outset of the project. This chapter presents a consolidated evaluation of the system's performance across its core functional domains, interface quality, and technical validation outcomes.

11.1 Functional Validation

The system has been validated across all primary operational modules:

- **Resource Administration:** The centralized CRUD engine manages library assets with strict referential integrity between books and categories, supporting atomic creation, modification, retrieval, and deletion across the full book catalog.
- **Identity and Access Management:** The stateless JWT security layer guarantees secure API communication, while granular RBAC enforces distinct operational boundaries for Administrators, Managers, and Users, as validated by both unit and integration tests.
- **Business Intelligence:** The real-time statistical dashboard provides administrators with immediate visibility into collection composition and system usage through Recharts-rendered visualizations.
- **Document Portability:** Automated generation of PDF and Excel reports operates correctly for variable dataset sizes, with streamed delivery ensuring acceptable response times under realistic load conditions.

11.2 Interface and User Experience

The React-based frontend delivers a fully responsive interface that adapts dynamically across desktop and mobile viewports. Navigation is managed through React Router's client-side routing, ensuring seamless transitions between modules without full-page reloads. Role-aware rendering ensures that administrative controls are displayed only to users whose JWT-encoded authorities permit the corresponding operations.

Figure 11.1: LibraCore Analytical Dashboard: Real-time administrative monitoring and collection statistics.

11.3 Technical Validation

The system architecture has been validated through the 37-test suite described in Chapter 9. The results confirm:

- Security configurations correctly prevent unauthorized data access, with the JWT filter chain rejecting requests that carry invalid, expired, or absent tokens.
- API responses are consistent and handle all error conditions gracefully through the centralized `GlobalExceptionHandler`, returning standardized error structures rather than raw server exceptions.
- Business logic—particularly in data export workflows and user management operations—remains stable under varied input conditions, including edge cases such as missing resources, duplicate identifiers, and malformed request payloads.

This implementation constitutes a scalable and maintainable foundation for modern library management, satisfying the predefined requirements for security, usability, and technical performance.

Chapter 12

Conclusion and Perspectives

This project has successfully delivered a robust, full-stack **Library Management System** that integrates modern web technologies into a cohesive, production-ready platform. The combination of React and Spring Boot achieves a fluid synergy between a responsive, user-centric frontend and a secure, performant RESTful backend, demonstrating that enterprise-grade library management can be implemented within an academic project scope.

12.1 Key Achievements

Throughout the development lifecycle, the project met several critical technical and architectural milestones:

- **Architectural Integrity:** The decoupled client-server architecture, enforced through a strict RESTful API boundary, ensures high maintainability and provides a clear path for independent scaling of the frontend and backend tiers.
- **Security Excellence:** The integration of JWT authentication and granular RBAC delivers enterprise-grade security, ensuring that each user class interacts only with the resources appropriate to their role, as rigorously verified by the integration test suite.
- **Operational Efficiency:** The addition of automated data export and real-time analytical dashboards elevates the system beyond a simple management tool, transforming it into a strategic asset that supports data-driven decision-making for library administrators.
- **Quality Assurance:** A 37-test suite spanning unit and integration levels validates the correctness of both the business logic and the API surface, providing a reliable regression baseline for future development.

12.2 Future Roadmap

While the current solution fulfills its defined requirements, several enhancements are identified as high-value directions for future development:

1. **Advanced Discovery:** Integration of an Elasticsearch engine to provide powerful full-text and fuzzy-search capabilities across the library catalog, significantly improving findability for large collections.
2. **Workflow Automation:** Implementation of a book reservation system coupled with automated email and SMS notification services to inform patrons of book availability and reservation status changes.
3. **Scalability and Deployment:** Containerization of the full application stack using Docker and orchestration via Kubernetes, enabling cloud deployment and ensuring high availability for institutional use at scale.

This project has deepened both technical expertise in the Java and React ecosystems and the practical ability to translate complex institutional requirements into high-quality, maintainable software solutions—competencies that form a strong foundation for professional engineering practice.